

COMP 4423: IMAGE PROCESSING 2

Easy Computer Vision

Xiaoyong Wei (魏驍勇)

x1wei@polyu.edu.hk



THE HONG KONG
POLYTECHNIC UNIVERSITY
香港理工大學

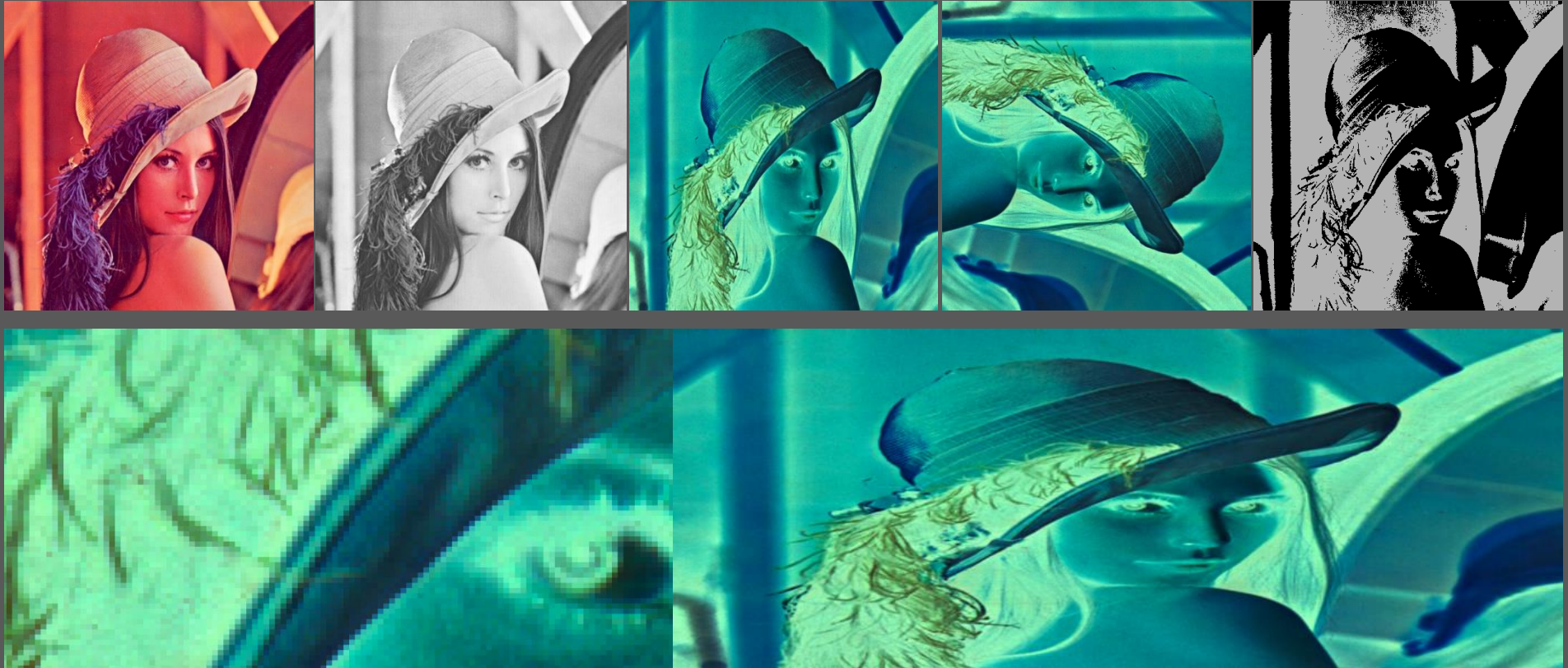
Opening Minds • Shaping the Future
啟迪思維 • 成就未來



How many coronavirus are there?



Image Processing 1





Those are for dealing with an image
as a whole.

Let's play the content!



Outline

- Filters and Convolutions
- Edge Filters
- Noise Reduction
- Morphological Operations



Edge Detection

Edges in image brightness may result from the change of

1. depths,
2. surface orientations,
3. materials,
4. and/or illumination.

https://en.wikipedia.org/wiki/Edge_detection/





Filters and Convolutions

a11	a12	a13	a14	a15	a16
a21	a22	a23	a24	a25	a26
a31	a32	a33	a34	a35	a36
a41	a42	a43	a44	a45	a46
a51	a52	a53	a54	a55	a56
a61	a62	a63	a64	a65	a66

6*6 image

*

1	0	-1
1	0	-1
1	0	-1

3*3 filter

=

4*4 image output after edge detection

a11	a12	a13	a14	a15	a16
a21	a22	a23	a24	a25	a26
a31	a32	a33	a34	a35	a36
a41	a42	a43	a44	a45	a46
a51	a52	a53	a54	a55	a56
a61	a62	a63	a64	a65	a66

6*6 image

*

1	0	-1
1	0	-1
1	0	-1

3*3 filter

=

4*4 image output after edge detection



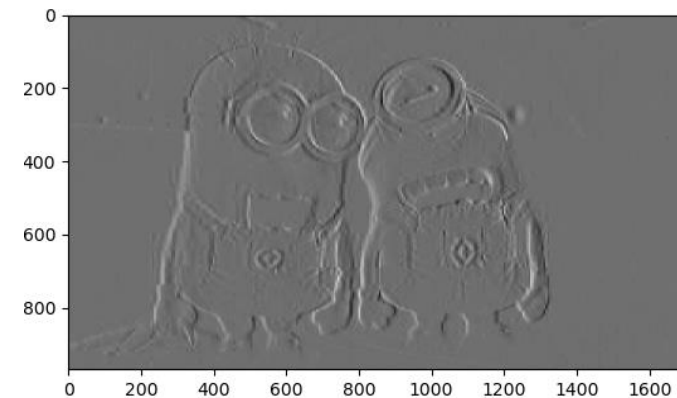
Prewitt Edge Detection

Prewitt is a commonly used edge detector for detecting the horizontal and vertical edges in images. Its edge filters are as follows:

$$G_x = \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix} \quad G_y = \begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ -1 & -1 & -1 \end{bmatrix}$$

Prewitt filter for vertical edge detection

Prewitt filter for horizontal edge detection





What is the rationale?



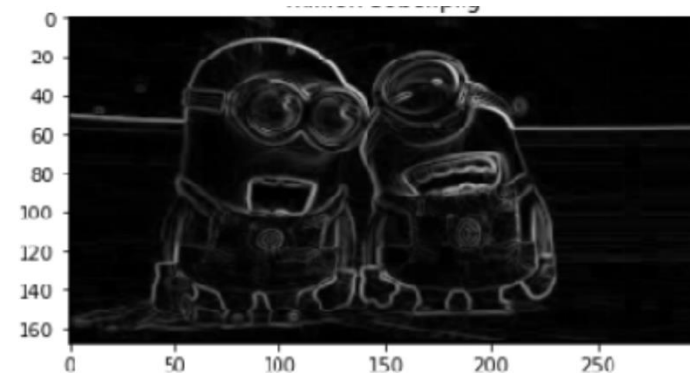
Sobel Edge Detection

The filter pays more attention to the center of the window. It is one of the most commonly used edge detectors, which reduces noise, differentiates regions, and responds to edges simultaneously.

$$G_x = \begin{bmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{bmatrix} \quad G_y = \begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$$

Sobel filter for vertical edge detection

Sobel filter for horizontal edge detection





Sobel Edge Detection

The filter pays more attention to the center of the window. It is one of the most commonly used edge detectors, which reduces noise, differentiates regions, and responds to edges simultaneously.

$$G_x = \begin{bmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{bmatrix} \quad G_y = \begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$$

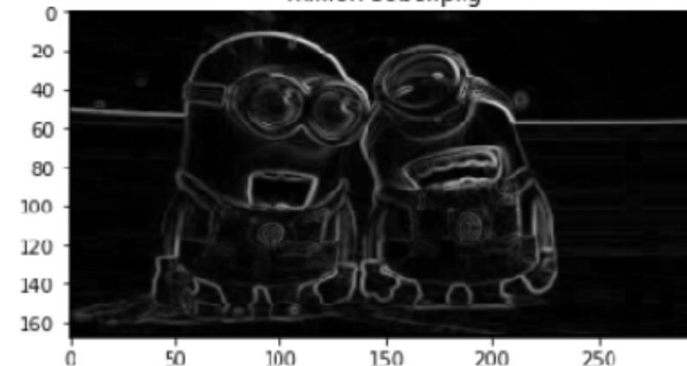
Sobel filter for vertical edge detection

Sobel filter for horizontal edge detection

Magnitude: $|G| = \sqrt{G_x^2 + G_y^2}$

Approximation: $|G| = |G_x| + |G_y|$

Orientation: $\theta = \arctan(G_y/G_x)$



<https://www.mygreatlearning.com/blog/introduction-to-edge-detection/>



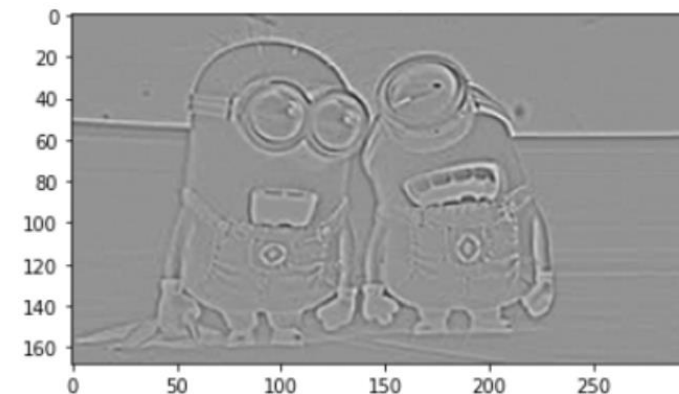
Laplacian Edge Detection

The Laplacian edge detectors uses only one filter. In a single pass, Laplacian edge detection performs second-order derivatives and hence are sensitive to noise. To avoid this sensitivity to noise, before applying this method, Gaussian smoothing can be done on the image.

-1	-1	-1
-1	8	-1
-1	-1	-1

0	-1	0
-1	4	-1
0	-1	0

Common Laplacian edge detection filters





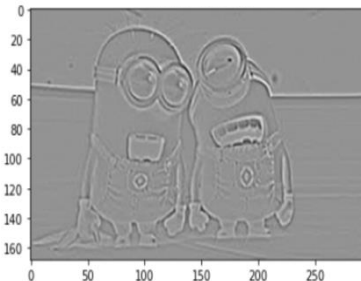
Laplacian Edge Detection

The Laplacian edge detectors use only one filter. In a single pass, Laplacian edge detection performs second-order derivatives and hence are sensitive to noise. To avoid this sensitivity to noise, before applying this method, Gaussian smoothing can be done on the image.

-1	-1	-1
-1	8	-1
-1	-1	-1

0	-1	0
-1	4	-1
0	-1	0

Common Laplacian edge detection filters



$$\begin{aligned}\nabla f_x(x, y) &= \frac{\partial f(x, y)}{\partial x} \\ &= \lim_{\epsilon \rightarrow 0} \frac{f(x, y) - f(x + \epsilon, y)}{\epsilon} \\ &\approx f(x, y) - f(x + 1, y)\end{aligned}$$

$$\nabla f^2(x, y) = \nabla f_x^2(x, y) + \nabla f_y^2(x, y)$$

$$\begin{aligned}\nabla f_x^2(x, y) &= \nabla f_x(x, y) - \nabla f_{x'}(x, y) \\ &\approx f(x, y) - f(x + 1, y) \\ &\quad - (f(x - 1, y) - f(x, y)) \\ &\approx -f(x - 1, y) + 2f(x, y) - f(x + 1, y) \\ \nabla f_y^2(x, y) &\approx -f(x, y - 1) + 2f(x, y) - f(x, y + 1)\end{aligned}$$



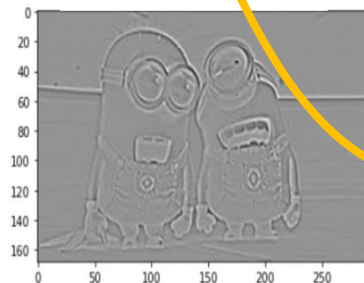
Laplacian Edge Detection

The Laplacian edge detectors use only one filter. In a single pass, Laplacian edge detection performs second-order derivatives and hence are sensitive to noise. To avoid this sensitivity to noise, before applying this method, Gaussian smoothing can be done on the image.

-1	-1	-1
-1	8	-1
-1	-1	-1

0	-1	0
-1	4	-1
0	-1	0

Common Laplacian edge detection filters



$$\begin{aligned}\nabla f_x(x, y) &= \frac{\partial f(x, y)}{\partial x} \\ &= \lim_{\epsilon \rightarrow 0} \frac{f(x, y) - f(x + \epsilon, y)}{\epsilon} \\ &\approx f(x, y) - f(x + 1, y)\end{aligned}$$

$$\begin{aligned}\nabla f^2(x, y) &= \nabla f_x^2(x, y) + \nabla f_y^2(x, y) \\ \nabla f_x^2(x, y) &= \nabla f_x(x, y) - \nabla f_{x'}(x, y) \\ &\approx f(x, y) - f(x + 1, y) \\ &\quad - (f(x - 1, y) - f(x, y)) \\ &\approx -f(x - 1, y) + 2f(x, y) - f(x + 1, y)\end{aligned}$$

$$\nabla f_y^2(x, y) \approx -f(x, y - 1) + 2f(x, y) - f(x, y + 1)$$

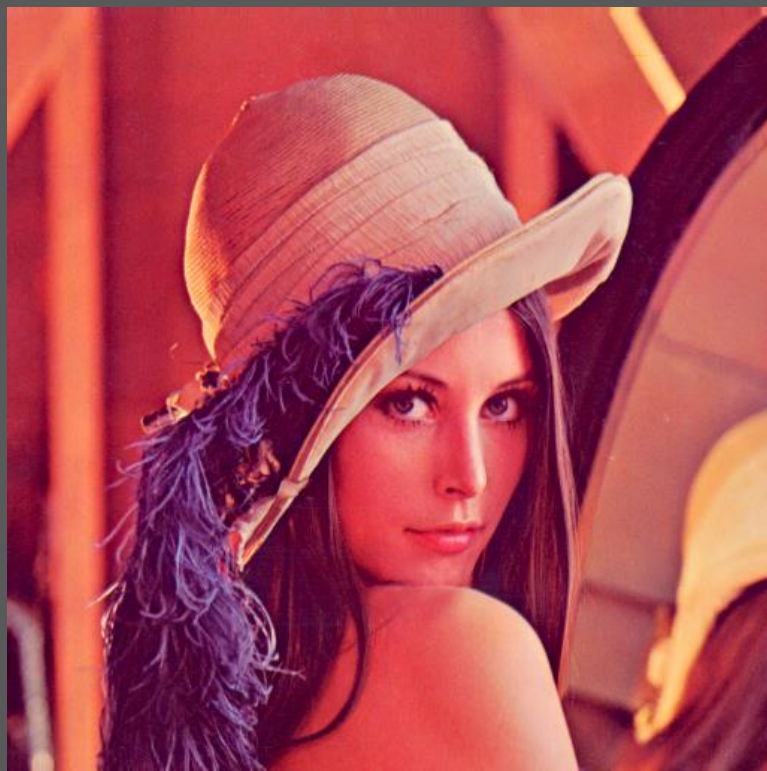




```
from PIL import Image, ImageFilter
import urllib.request
import skimage.io

# save the image from the url
urllib.request.urlretrieve('https://upload.wikimedia.org/wikipedia/en/7/7d/Lenna_%28test_image%29.png',
                           'lenna.png')

# load the image using PIL.Image
img_lenna = Image.open('lenna.png')
# convert the image into gray scale
img_lenna_gray = img_lenna.convert("L")
# detect the edges using the argumented filter
image_edge = img_lenna_gray.filter(ImageFilter.FIND_EDGES)
display(image_edge)
```





```
# detect the edges using the Laplacian filter
image_edge = img_lenna_gray.filter(ImageFilter.Kernel((3, 3), (-1, -1, -1, -1, 8,
                                                                -1, -1, -1, -1), 1, 0))
display(image_edge)
```





```
# detect the edges using the Sobel filters
image_edge_x = img_lenna_gray.filter(ImageFilter.Kernel((3, 3), (1, 2, 1, 0, 0,
                                                                0, -1, -2, -1), 1, 0))

display(image_edge_x)
image_edge_y = img_lenna_gray.filter(ImageFilter.Kernel((3, 3), (1, 0, -1, 2, 0,
                                                                -2, 1, 0, -1), 1, 0))

display(image_edge_y)
```



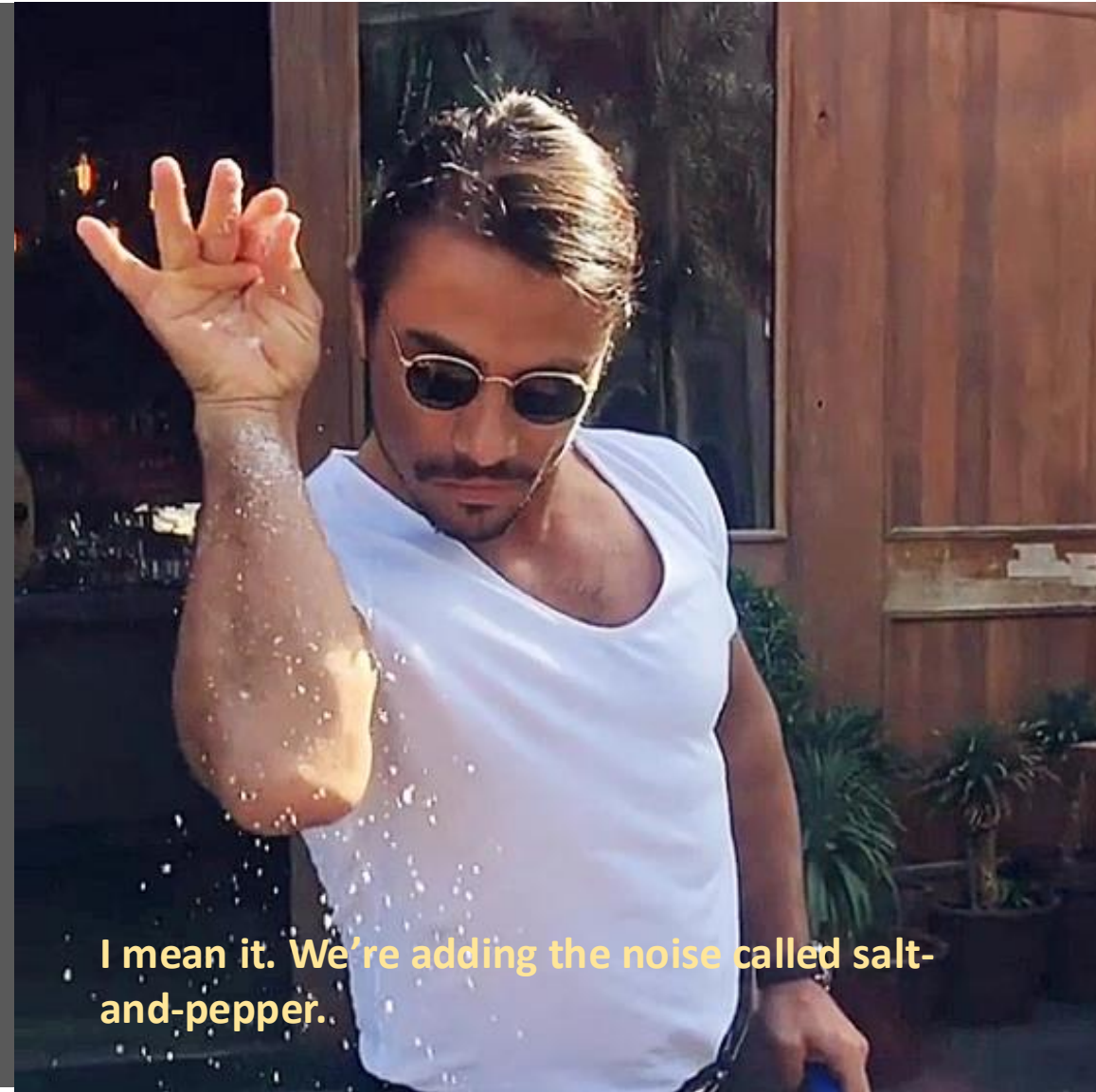


Filters is a useful tool not only for
Edge Detection.

To begin with, let's try Noise
Reduction!



But we have to add some “salt” first



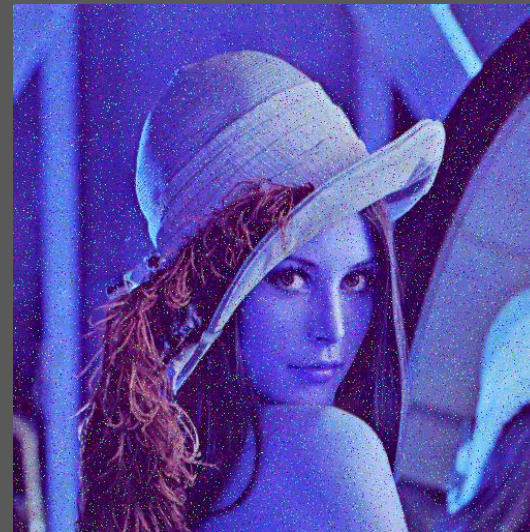
I mean it. We're adding the noise called salt-and-pepper.



But we have to add some noise first

```
from skimage.util import random_noise
import cv2
import numpy as np

# load the image
img_lenna = cv2.imread('lenna.png')
# Add salt-and-pepper noise
lenna_noise_array = random_noise(img_lenna, mode='s&p', amount=0.05)
print(type(lenna_noise_array))
print((lenna_noise_array.dtype))
# we have to convert the array into uint8 before composing the image
lenna_noise_array = np.array(255*lenna_noise_array, dtype = 'uint8')
lenna_noise_img=Image.fromarray(lenna_noise_array)
display(lenna_noise_img)
```





Mean Filter

The idea of the Mean Filter is to “average out” the noise using the surrounding pixels. However, it will “blur” the normal pixels at the same time.

$\frac{1}{9}$	$\frac{1}{9}$	$\frac{1}{9}$
$\frac{1}{9}$	$\frac{1}{9}$	$\frac{1}{9}$
$\frac{1}{9}$	$\frac{1}{9}$	$\frac{1}{9}$



Mean Filter



```
image_denoise = lenna_noise_img.filter(ImageFilter.Kernel((3, 3), (1/9, 1/9, 1/9, 1/9, 1/9, 1/9, 1/9, 1/9, 1/9), 1, 0))  
display(image_denoise)
```





Median Filter

Instead of using the average-out, the Median filters reduce the noise by “replacing” the central pixel by the median of pixels in the neighborhood. The assumption is that noise appear as the “outliers” on which the *median* works better than the *mean*.

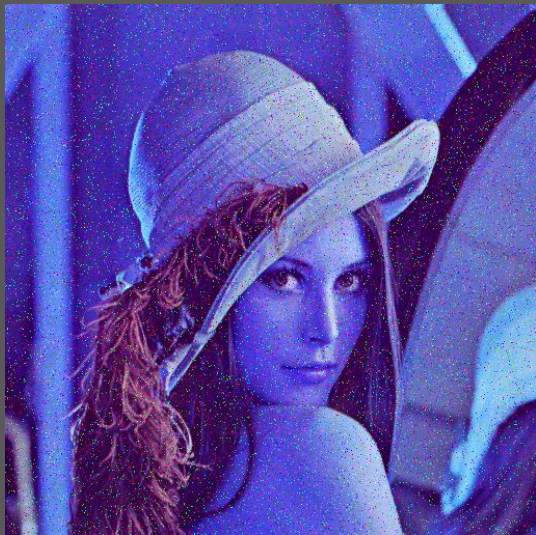
	10	5	20					
	14	80	11					
	8	3	22					

(3,5,8,10,11,14,20,22,80)



Mean and Median Filters

```
image_denoise = lenna_noise_img.filter(ImageFilter.MedianFilter)  
display(image_denoise)
```



Mean Filter



Median Filter



Noise vs. Filters

Median filters seem worked better than Mean filters for salt-and-pepper noise. However, it doesn't mean it's a better choice in a general sense. The performance of filters are dependent on the type of noise. Please find below the noise and filters and try them out by yourself.

Noise	Recognized Filters
Salt and pepper	Median
Poisson	Mean
Gaussian	Gaussian
Speckle	Weiner

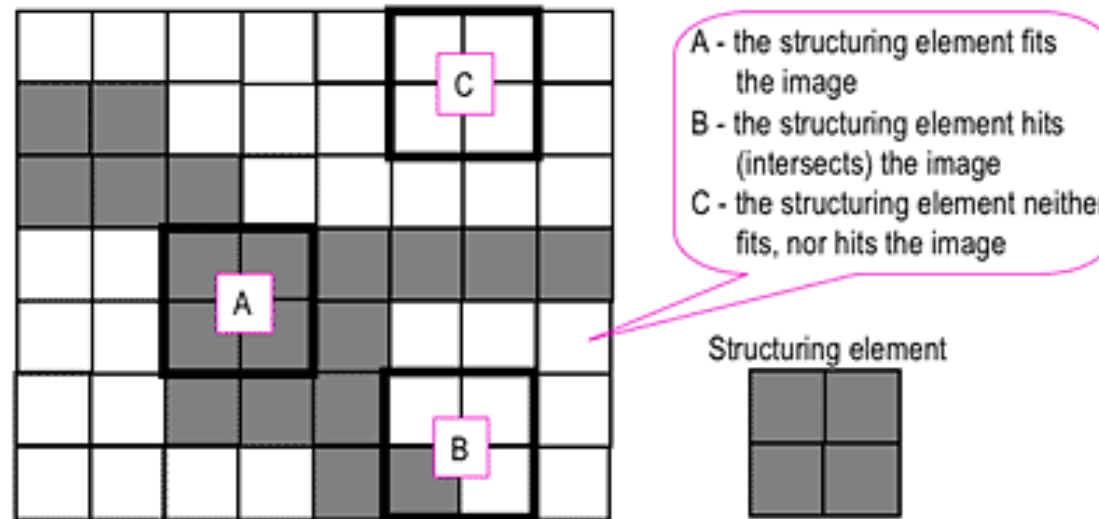


Another set of tools, which is similar
to Filters, is
Morphological Operations.



Morphological Operations

Morphological techniques probe an image with a small shape or template called a **Structuring Element (SE)**. The structuring element is positioned at all possible locations in the image and it is compared with the corresponding neighborhood of pixels. Some operations test whether the element "fits" within the neighborhood, while others test whether it "hits" or intersects the neighborhood:





Structuring elements can be considered as filters with which a different way of filtering (i.e., Morphological Operations) is defined (rather than using the weighted averages).

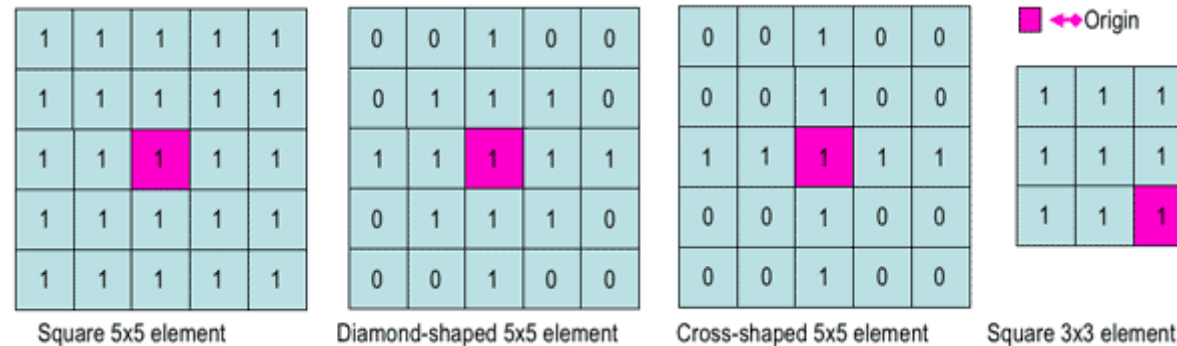


In addition, it is usually applied to the **binary images**, on which it's easier to determine the fitness of a structuring element to image positions.



Morphological Operations

- > The **structuring element** is a small binary image, i.e. a small matrix of pixels, each with a value of zero or one:
- The matrix dimensions specify the size of the structuring element.
 - The pattern of ones and zeros specifies the shape of the structuring element.
 - An origin of the structuring element is usually one of its pixels, although generally the origin can be outside the structuring element.



A common practice is to have odd dimensions of the structuring matrix and the origin defined as the centre of the matrix. Structuring elements play in morphological image processing the same role as convolution kernels in linear image filtering.



Morphological Operations

> **Fit:**

- The structuring element is said to **fit** the image if, **for each of** its pixels set to 1, the corresponding image pixel is also 1.
- $sum(Region \cap S) = sum(S)$

> **Hit:**

- Similarly, a structuring element is said to **hit**, or intersect, an image if, **at least for one of** its pixels set to 1 the corresponding image pixel is also 1.
- $sum(Region \cap S) > 0$

Diagram illustrating morphological operations. A 10x10 binary image is shown with three regions of interest highlighted by dashed pink boxes: Region A (bottom-left), Region B (top-left), and Region C (top-right). To the right, two structuring elements are defined: $s_1 = \begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{pmatrix}$ and $s_2 = \begin{pmatrix} 0 & 1 & 0 \\ 1 & 1 & 1 \\ 0 & 1 & 0 \end{pmatrix}$. Further right, a table summarizes the 'fit' and 'hit' results for these structuring elements across the three regions.

		A	B	C
fit	s_1	yes	no	no
	s_2	yes	yes	no
hit	s_1	yes	yes	yes
	s_2	yes	yes	no



Morphological Operations

> Basic morphological operations include erosion, dilation, opening and closing.

> **Erosion**

> The erosion of a binary image f by a structuring element s (denoted $f \ominus s$) produces a new binary image $g = f \ominus s$ with ones in all locations (x, y) of a structuring element's origin at which that structuring element s **fits** the input image f , i.e. $g(x, y) = 1$ if s fits f and 0 otherwise, repeating for all pixel coordinates (x, y) .



Greyscale image



Binary image by thresholding



Erosion: a 2×2 square structuring element

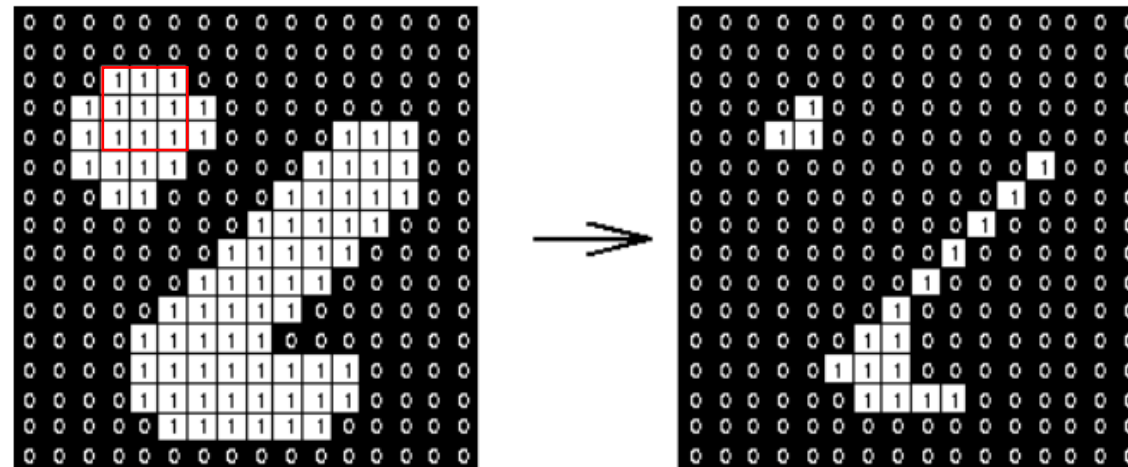
<https://www.cs.auckland.ac.nz/courses/compsci773s1c/lectures/ImageProcessing-html/topic4.htm>

<http://documents.wolfram.com/applications/digitalimage/UsersGuide/Morphology/ImageProcessing6.3.html>



Morphological Operations

Erosion with small (e.g. 2×2 - 5×5) square structuring elements shrinks an image by stripping away a layer of pixels from both the inner and outer boundaries of regions.

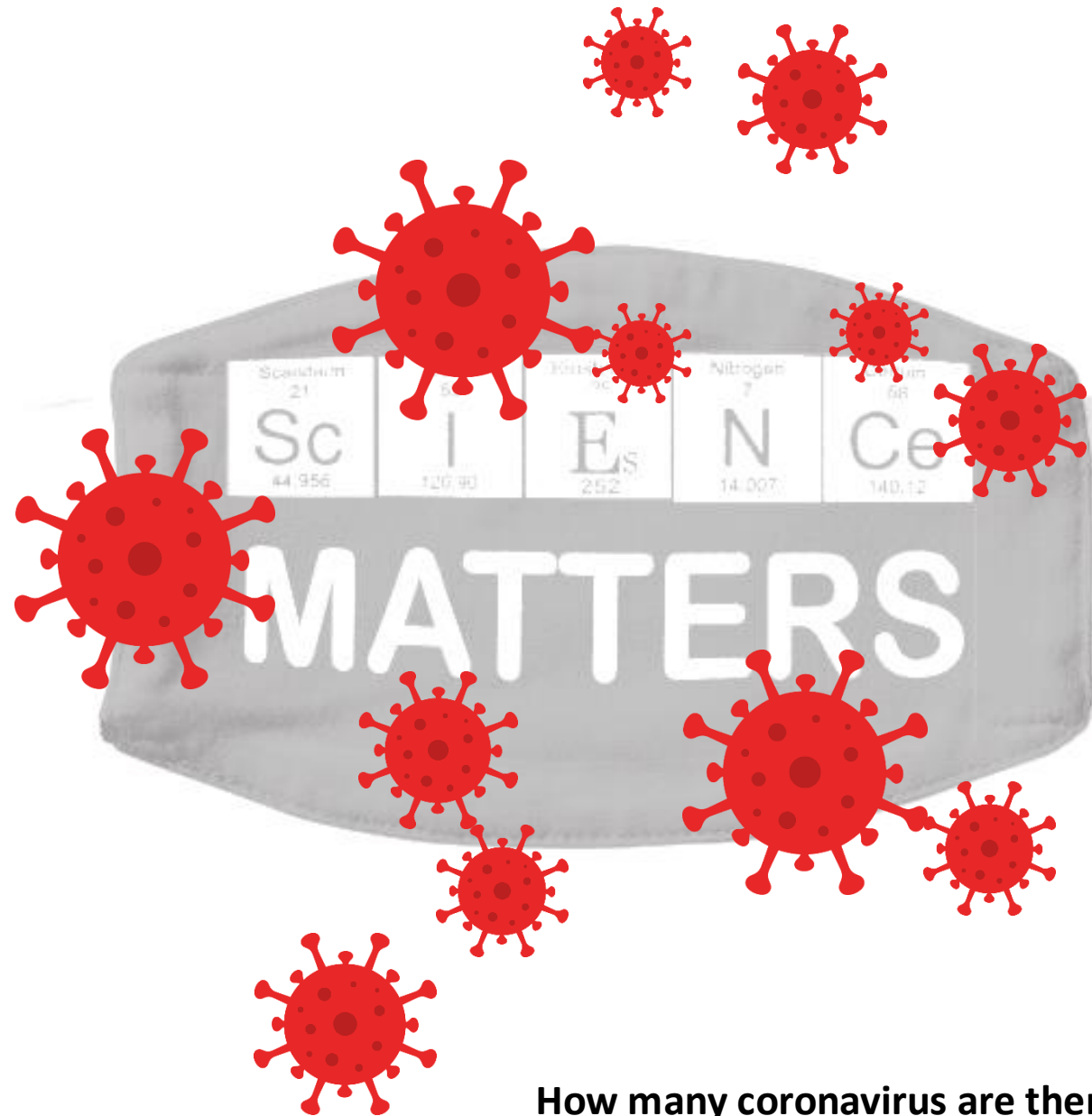


Erosion: a 3×3 square structuring element

Erosion removes small-scale details from a binary image but simultaneously reduces the size of regions of interest, too. By subtracting the eroded image from the original image, boundaries of each region can be found: $b = f - (f \ominus s)$ where f is an image of the regions, s is a 3×3 structuring element, and b is an image of the region boundaries.



Let's learn this through a new
example of
“Find the Coronaviruses”

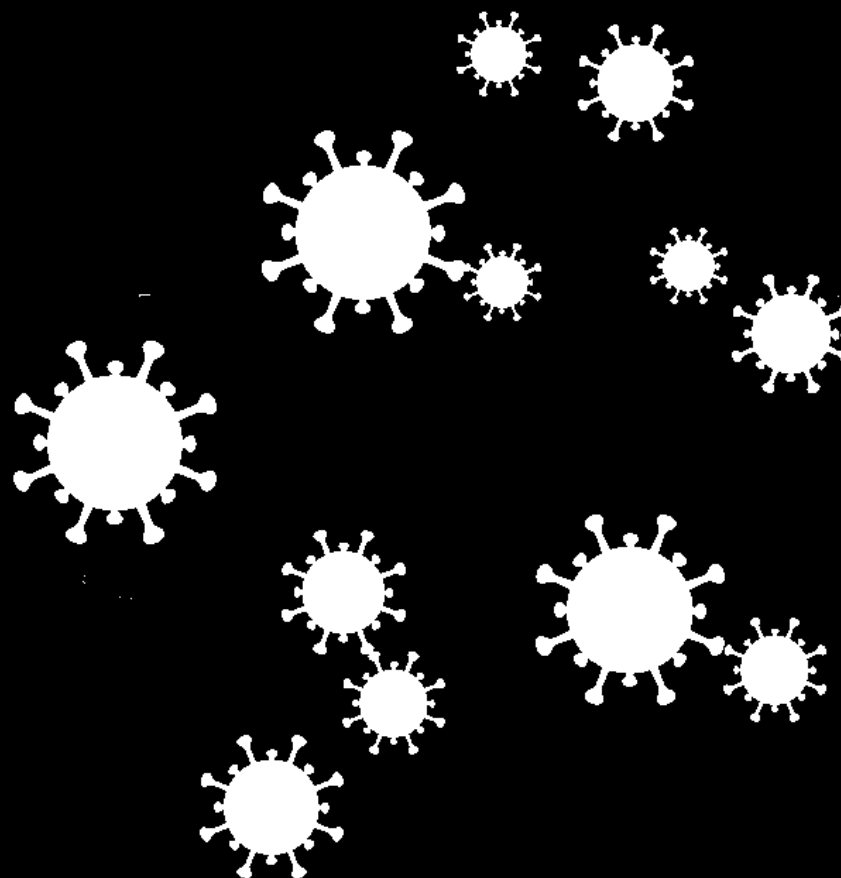


How many coronavirus are there?



```
import cv2
```

```
# load the image containing coronavirus
img_virus=cv2.imread("coronavirus-mask.png")
# convert to grey image
img_vir_grey = cv2.cvtColor(img_virus, cv2.COLOR_BGR2GRAY)
# get the binary image after thresholding
ret,img_th = cv2.threshold(img_vir_grey,160,255,cv2.THRESH_BINARY)
bw_img=Image.fromarray(255-img_th)
display(bw_img)
```

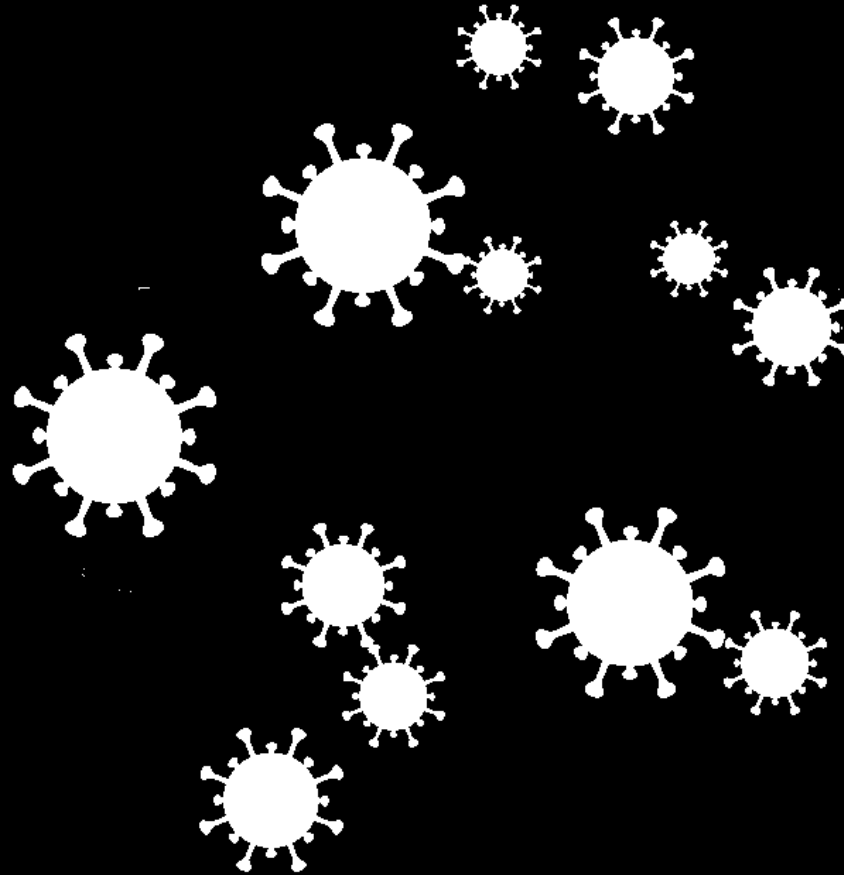




```
import cv2

# load the image containing coronavirus
img_virus=cv2.imread("coronavirus-mask.png")
# convert to grey image
img_vir_grey = cv2.cvtColor(img_virus, cv2.COLOR_BGR2GRAY)
# get the binary image after thresholding
ret,img_th = cv2.threshold(img_vir_grey,160,255,cv2.THRESH_BINARY)
bw_img=Image.fromarray(255-img_th)
display(bw_img)
```

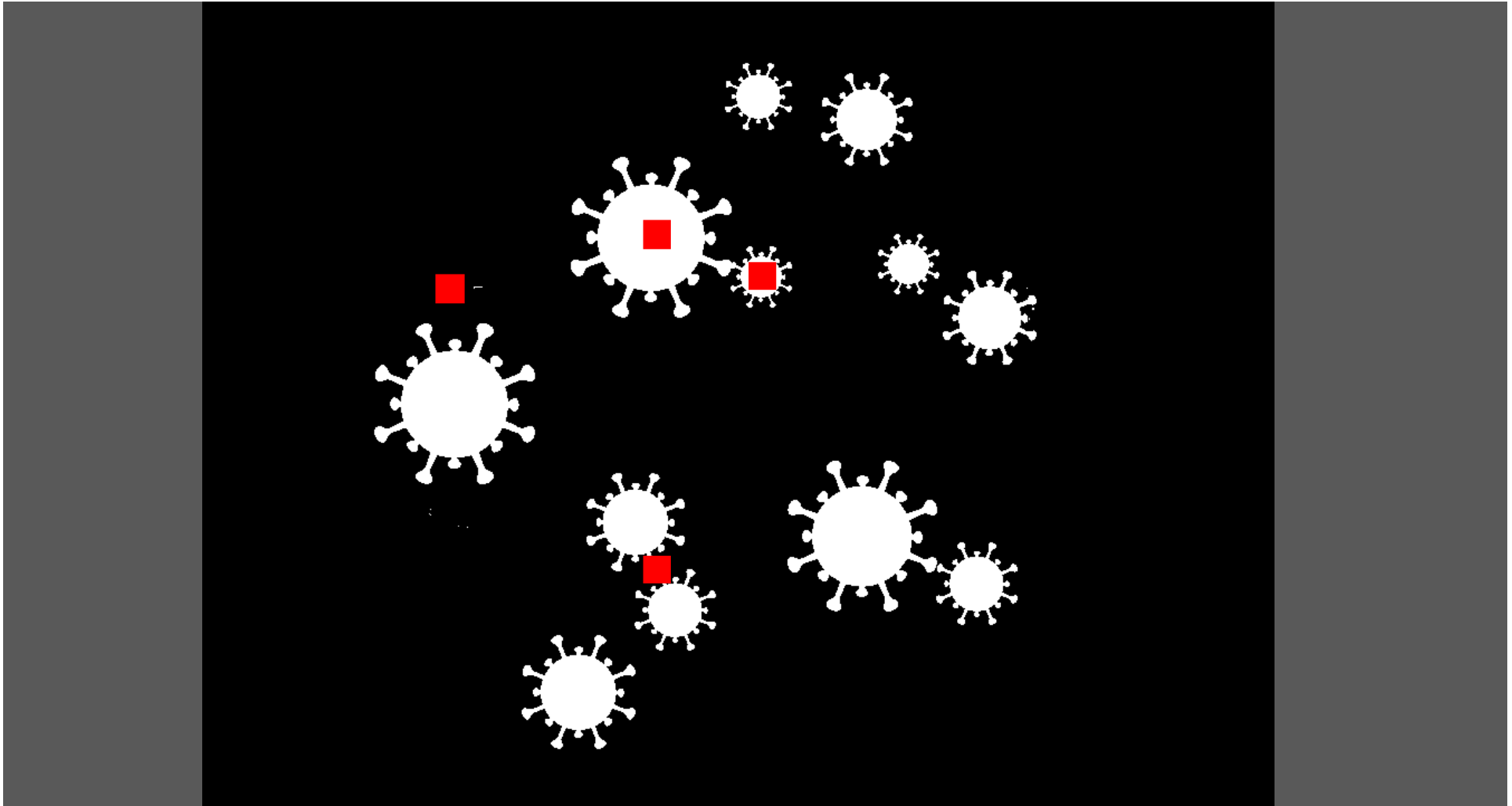
Looks good! But there are noise and connected components, which distract the computers from counting the virus precisely.





Use Erosion to exclude the distractions

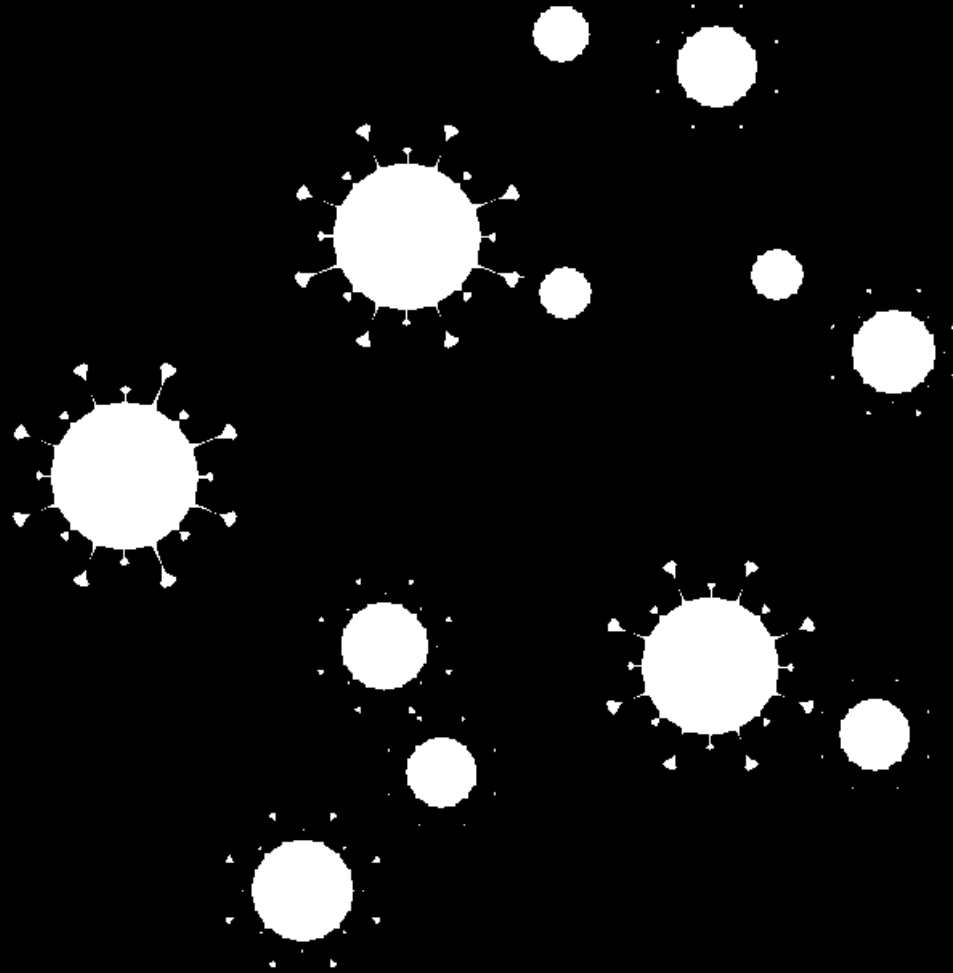
- > **Erosion** excludes the positions that the Structing Element (SE) dose not fits. We can design a SE that fits into the bodies of viruses but not the noise and connections.





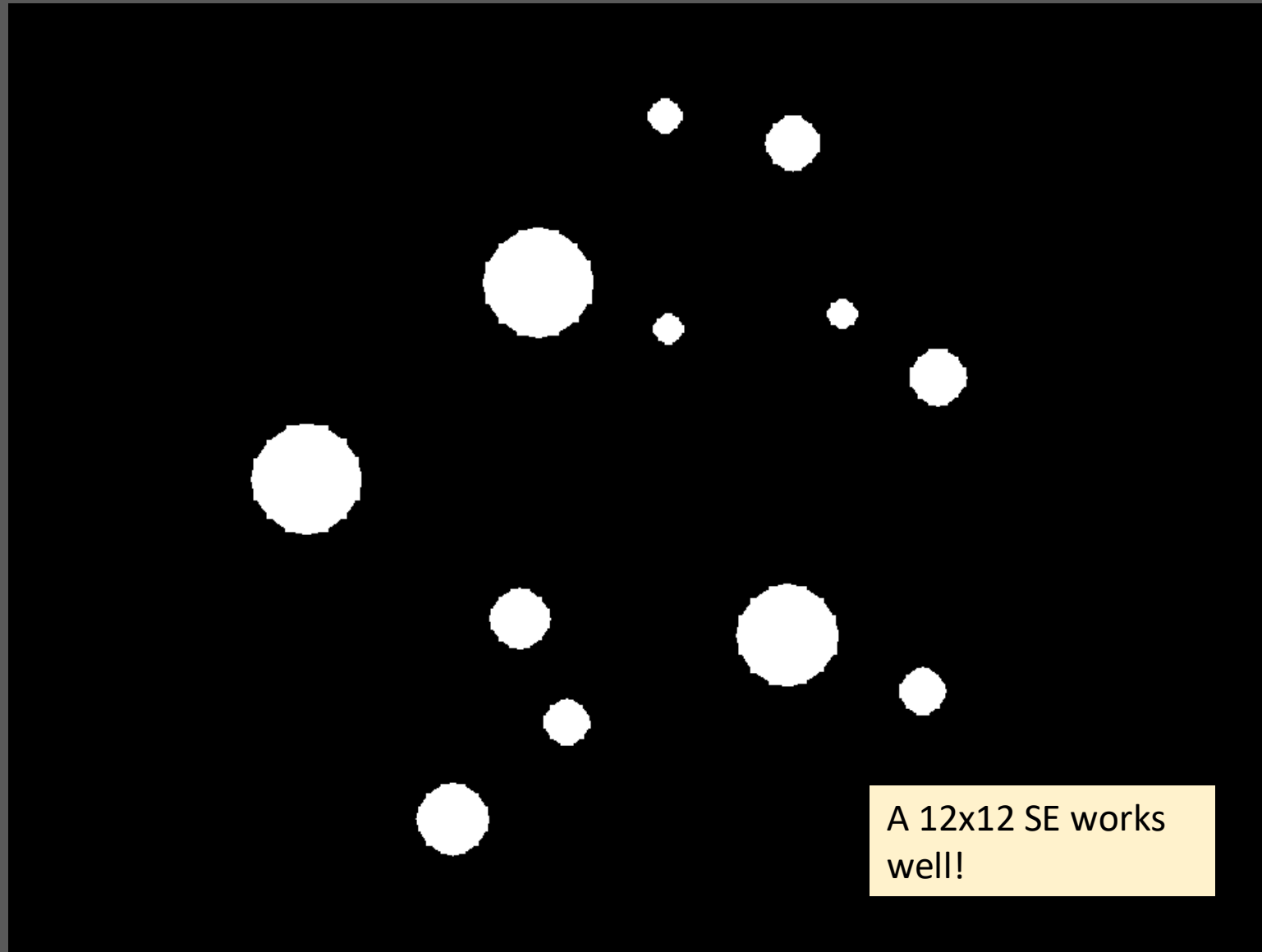
```
import numpy as np
#define a 5x5 SE
SE = np.ones((5,5),np.uint8)
img_erosion = cv2.erode(255-img_th,SE,iterations = 1)
bw_img=Image.fromarray(img_erosion)
display[bw_img]
```

The 5x5 SE does exclude the noise and break the connected components, but isolates the Spike Glycoproteins from the body at the same time.





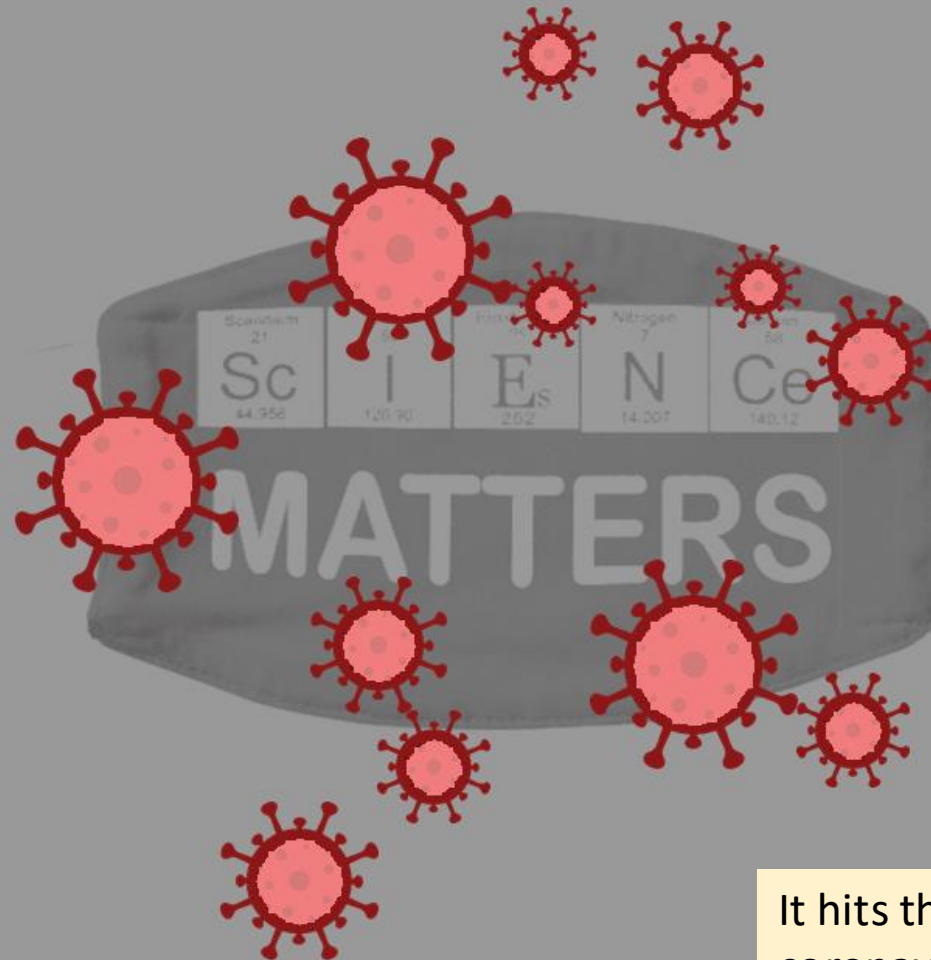
```
SE = np.ones((12,12),np.uint8)
img_erosion = cv2.erode(255-img_th,SE,iterations = 1)
bw_img=Image.fromarray(img_erosion)
display(bw_img)
```



A 12x12 SE works well!



```
# let's merge it back to the original image and see the result of localization  
np_merge=cv2.merge((img_erosion,img_erosion,img_erosion))*0.4+np_virus*0.6  
display(Image.fromarray(cv2.cvtColor(np_merge.astype(np.uint8), cv2.COLOR_BGR2RGB)))
```



It hits the
coronavirus well!



Now, we can count the viruses by the number of the white dots

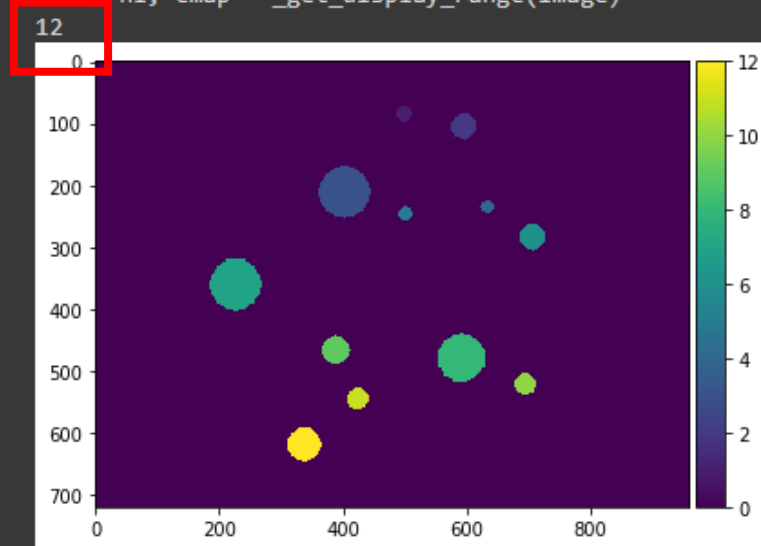


The white dots hit the virus well, which means we can count the virus by the number of dots.

This can be done using the *skimage.measure.label* and *skimage.measure.regionprops*.

```
from skimage.measure import label, regionprops
from skimage import io
np_labeled=label(img_erosion)
io.imshow(np_labeled)
regions = regionprops(np_labeled)
print(len(regions))
```

```
/usr/local/lib/python3.7/dist-packages/skimage/io/_plugins/matplotlib_plugin.py:150: UserWarning: Low image data range; displaying :
lo, hi, cmap = _get_display_range(image)
```



There are 12 dots (viruses)!



Studies show that the bigger a virus
is, the more infectious it is.
(I made it up. It's not necessarily true.)

Can you measure how infectious
each of the virus is?



We can measure the area of each virus. Let's build a mask for each of them first. This can be done using **Dilation.**



Morphological Operations

- > **Dilation:** The dilation of an image f by a structuring element s (denoted $f \oplus s$) produces a new binary image $g = f \oplus s$ with ones in all locations (x, y) of a structuring element's origin at which that structuring element s **hits** the the input image f , i.e. $g(x, y) = 1$ if s hits f and 0 otherwise, repeating for all pixel coordinates (x, y) .



Binary image



Dilation: a 2x2 square structuring element

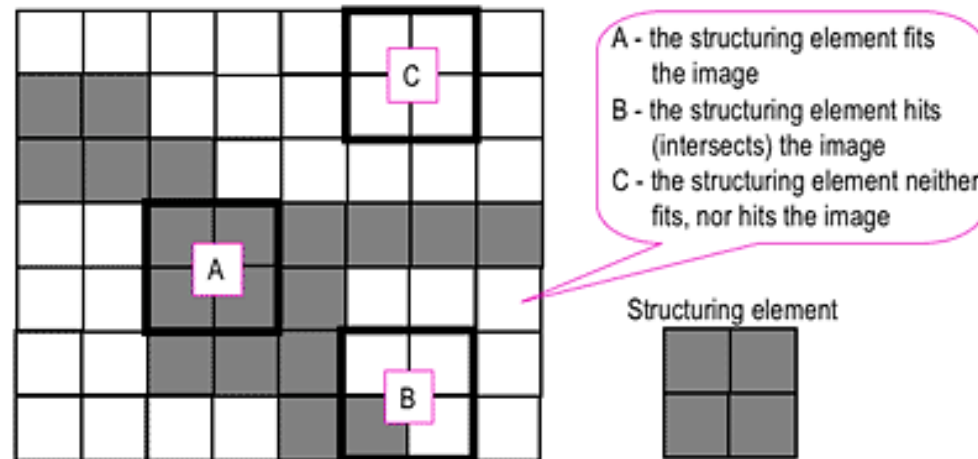
<https://www.cs.auckland.ac.nz/courses/compsci773s1c/lectures/ImageProcessing-html/topic4.htm>

<http://documents.wolfram.com/applications/digitalimage/UsersGuide/Morphology/ImageProcessing6.3.html>



Morphological Operations

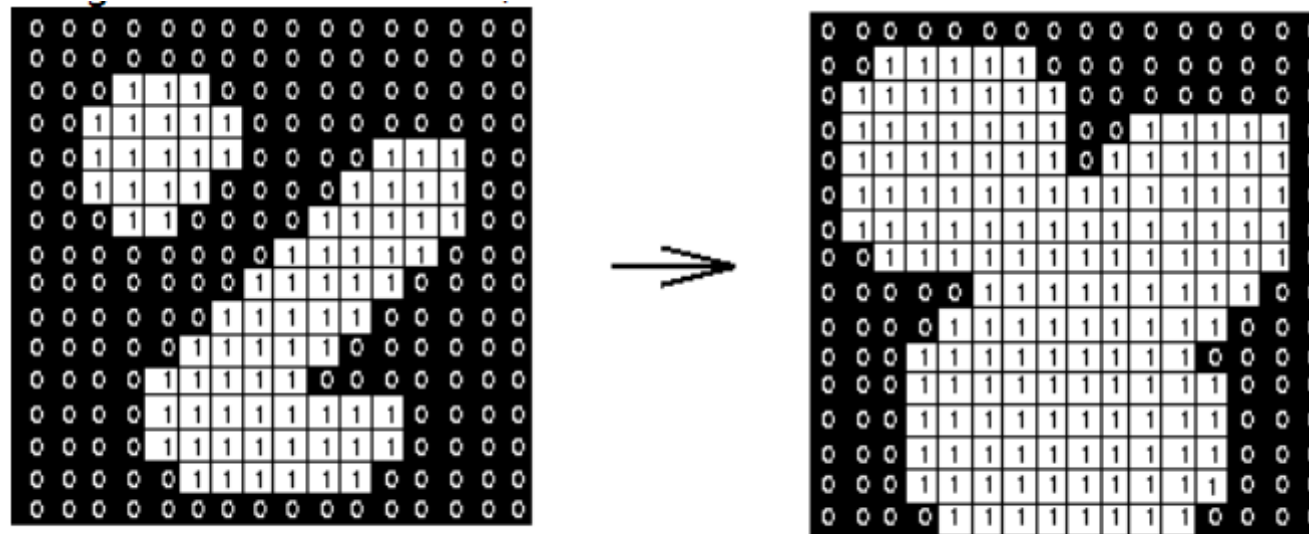
Morphological techniques probe an image with a small shape or template called a structuring element. The structuring element is positioned at all possible locations in the image and it is compared with the corresponding neighborhood of pixels. Some operations test whether the element "fits" within the neighborhood, while others test whether it "hits" or intersects the neighborhood:





Morphological Operations

- > Dilation has the opposite effect to erosion -- it adds a layer of pixels to both the inner and outer boundaries of regions.

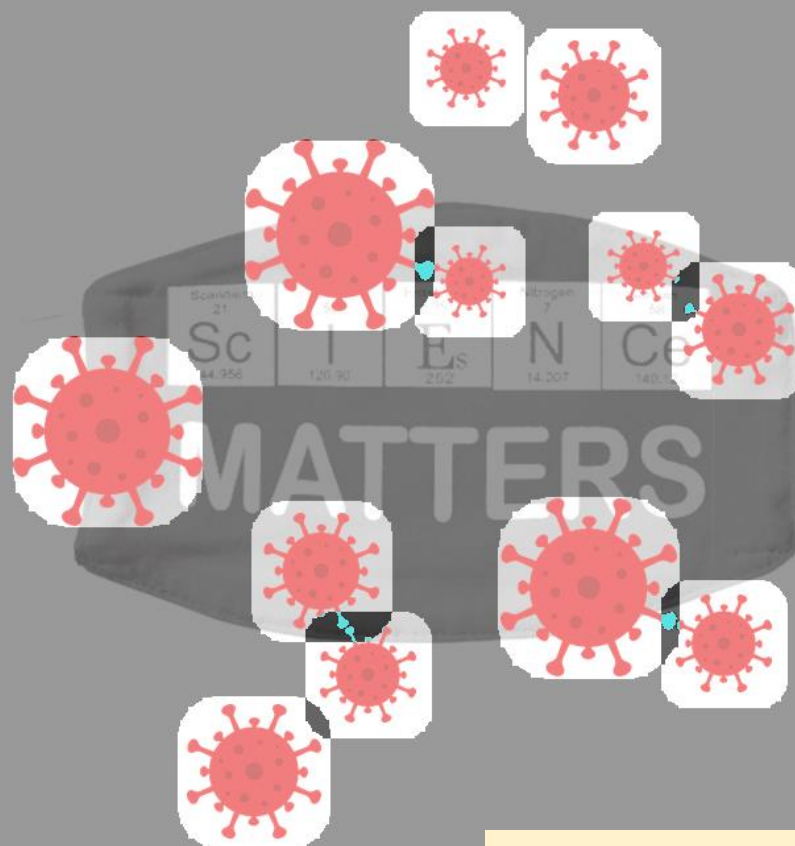


Dilation: a 3×3 square structuring element



```
np_regions=[]
np_masks=[]
merged_mask=np.zeros(np_labeled.shape)
for index in range(1, np_labeled.max()+1):
    np_regions.append((((np_labeled==index)+0)*255).astype(np.uint8))
    np_dilation = cv2.dilate(np_regions[-1],SE,iterations = 15)
    np_masks.append(np_dilation)
    merged_mask=merged_mask+np_dilation
    #display(Image.fromarray(np_dilation))

np_merge=cv2.merge((merged_mask,merged_mask,merged_mask))*0.4+np_virus*0.6
display(Image.fromarray(cv2.cvtColor(np_merge.astype(np.uint8), cv2.COLOR_BGR2RGB)))
```



The masks cover the viruses well

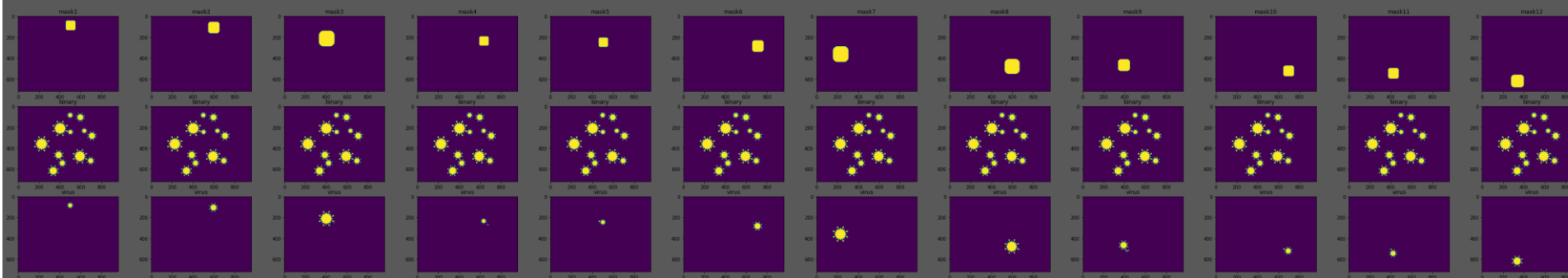


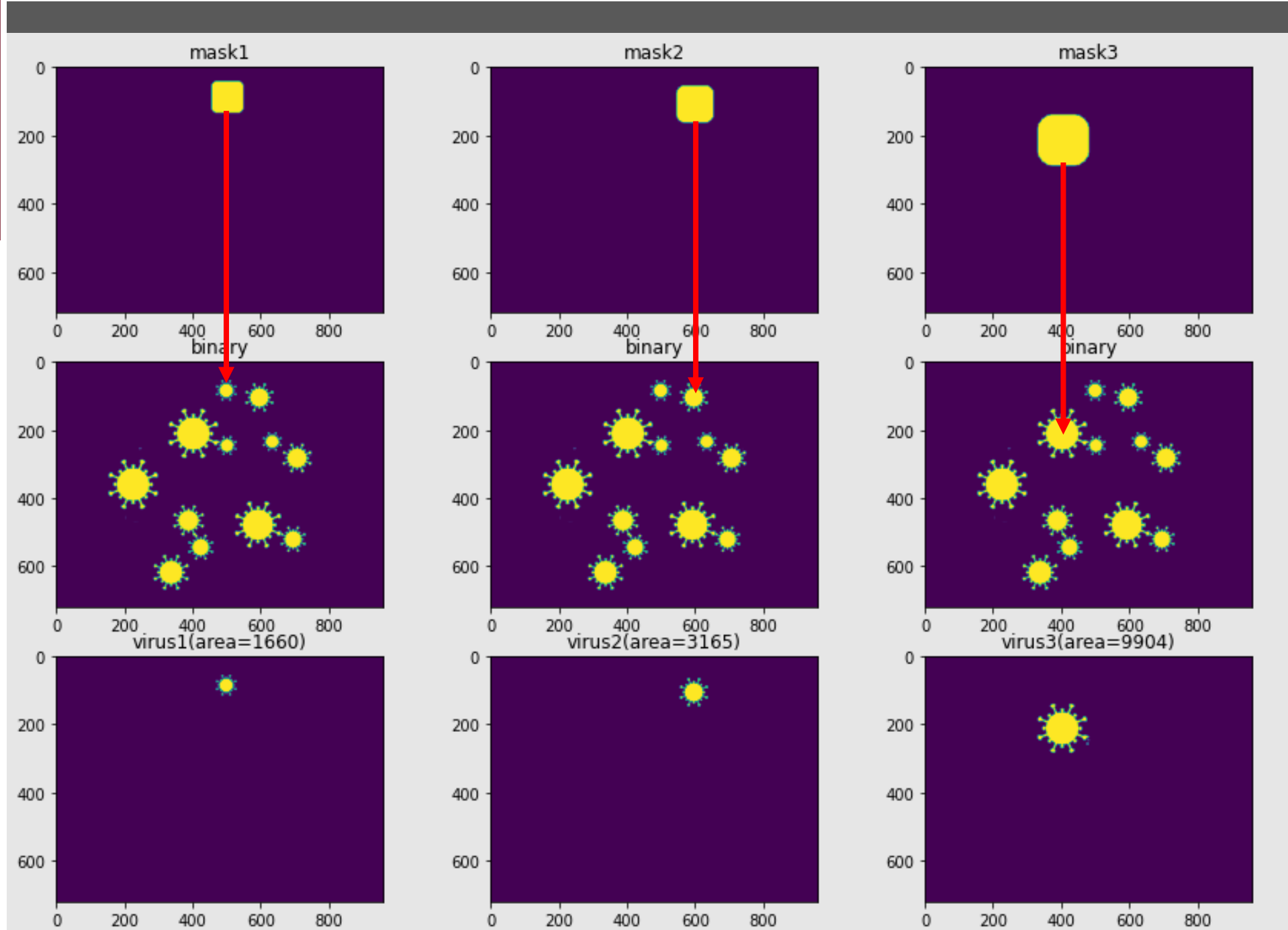
Now we can use the masks to separate the viruses from each other. This can be done by calculating the **AND** of a mask and the original binary image.

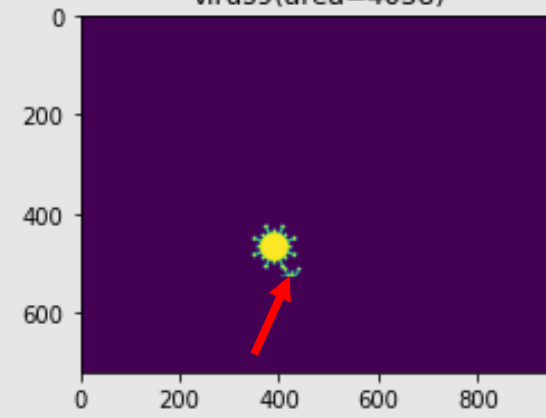
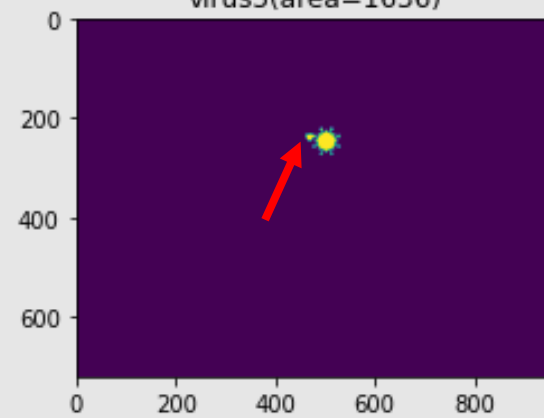
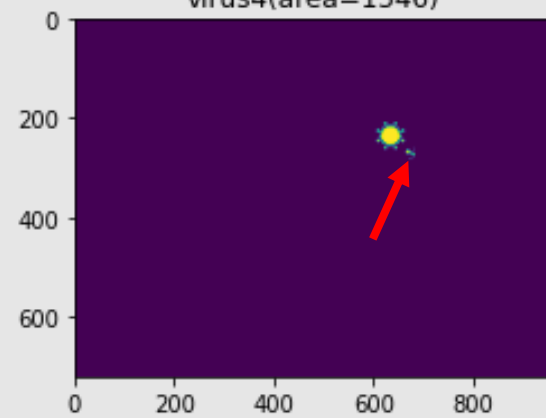
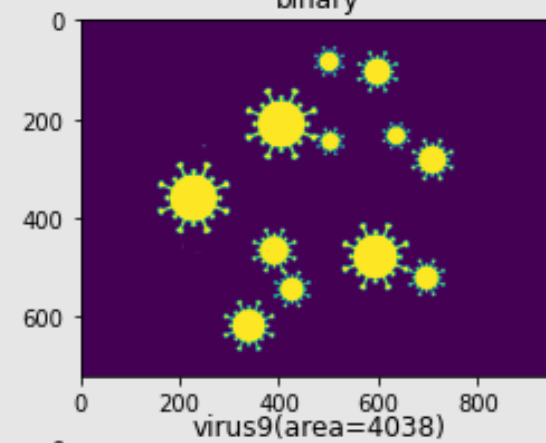
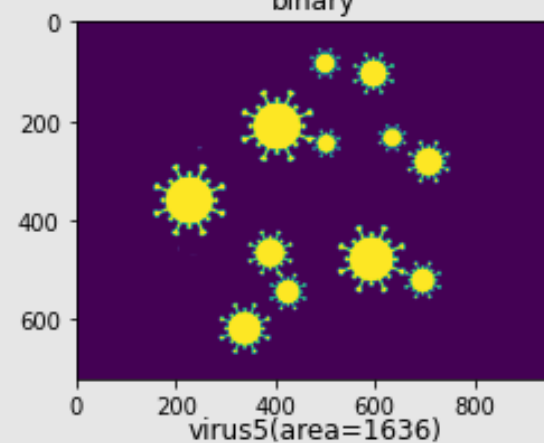
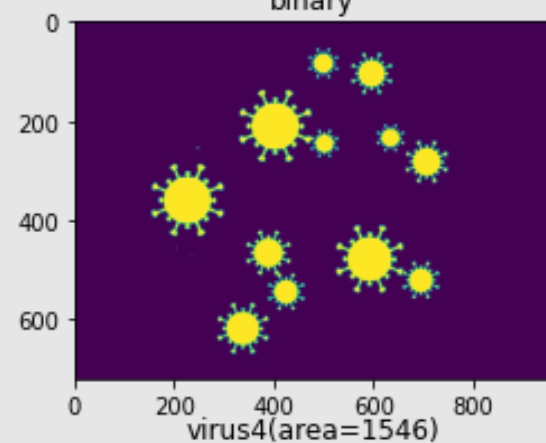
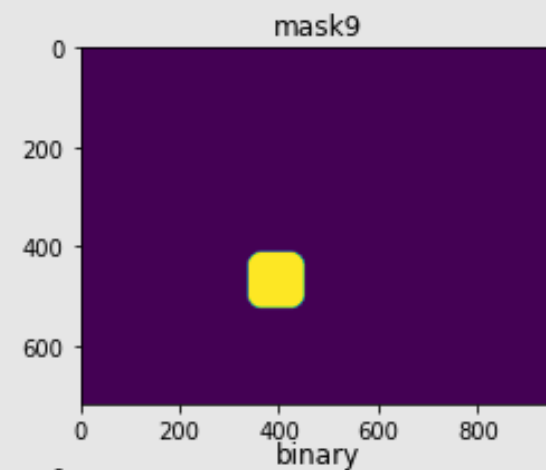
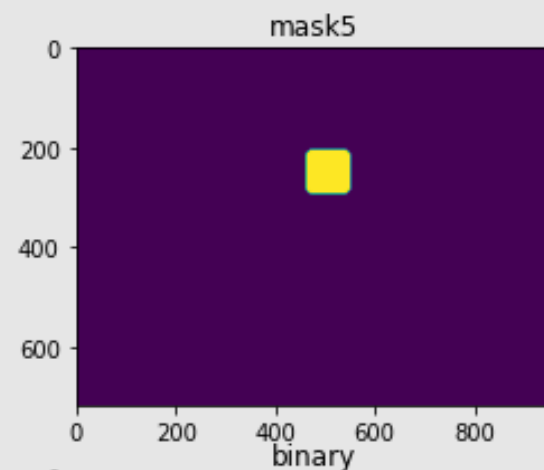
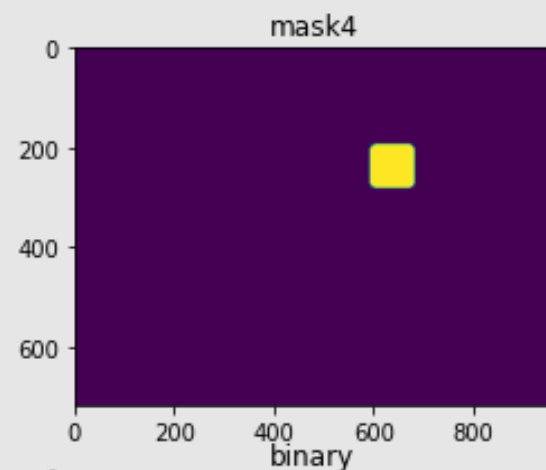


```
import matplotlib.pyplot as plt
fig = plt.figure(figsize=(60, 10))
binary_np=255-img_th
idx=1
plts=[]
for mask in np_masks:
    binary_virus=((mask==255)*(binary_np==255))
    area=np.sum(binary_virus)
    #display(Image.fromarray(cv2.cvtColor((binary_virus*255).astype(np.uint8), cv2.COLOR_GRAY2RGB)))
    plts.append(fig.add_subplot(3, 12, idx))
    plts[-1].set_title('mask'+str(idx))
    plt.imshow(mask)
    plts.append(fig.add_subplot(3, 12, idx+12))
    plts[-1].set_title('binary')
    plt.imshow(binary_np)
    plts.append(fig.add_subplot(3, 12, idx+24))
    plts[-1].set_title('virus')
    plt.imshow(binary_virus)
    print('Area of virus '+str(idx)+':\t'+str(area))
    idx=idx+1
```

Area of virus 1:	1660
Area of virus 2:	3165
Area of virus 3:	9904
Area of virus 4:	1546
Area of virus 5:	1636
Area of virus 6:	3453
Area of virus 7:	9912
Area of virus 8:	8716
Area of virus 9:	4038
Area of virus 10:	2666
Area of virus 11:	2607
Area of virus 12:	4877









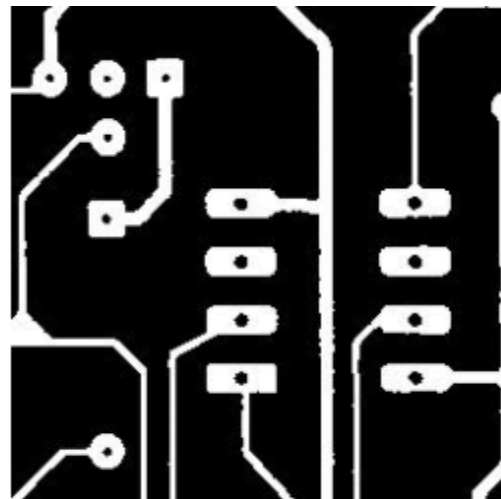
Not perfect, but OK.

Is there a better way?

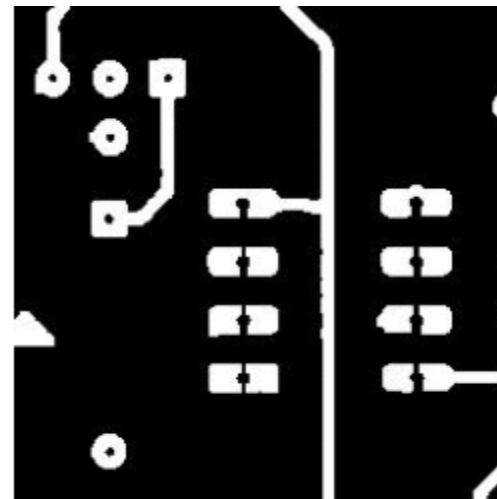


Morphological Operations

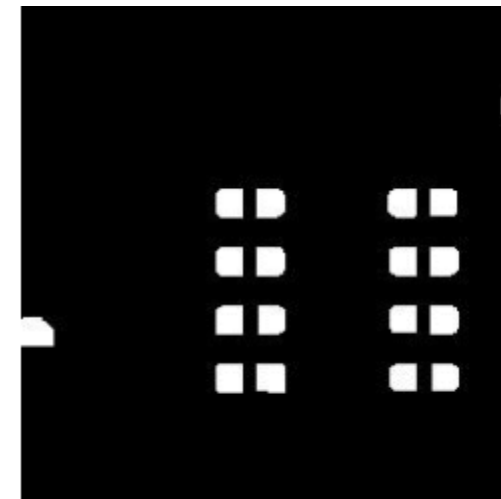
- > **Opening:** The opening of an image f by a structuring element s (denoted by $f \circ s$) is **an erosion followed by a dilation**:
- > $f \circ s = (f \ominus s) \oplus s$
- > Opening is so called because it can open up a gap between objects connected by a thin bridge of pixels. Any regions that have survived the erosion are restored to their original size by the dilation:



Binary image f



$f \circ s$ (5×5 square)

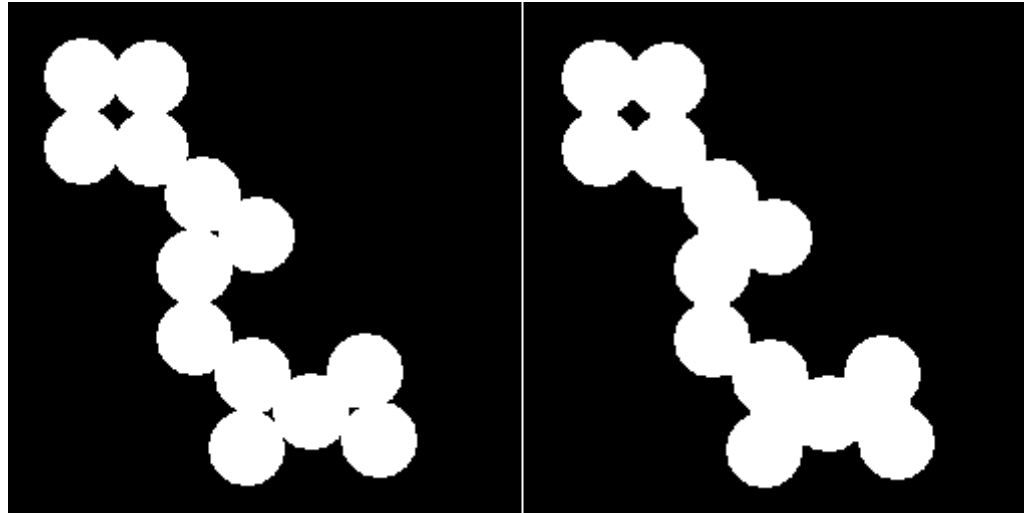


$f \circ s$ (9×9 square)



Morphological Operations

- > **Closing:** The closing of an image f by a structuring element s (denoted by $f \bullet s$) is a **dilation followed by an erosion**:
- > $f \bullet s = (f \oplus s) \ominus s$
- > Closing is so called because it can fill holes in the regions while keeping the initial region sizes.





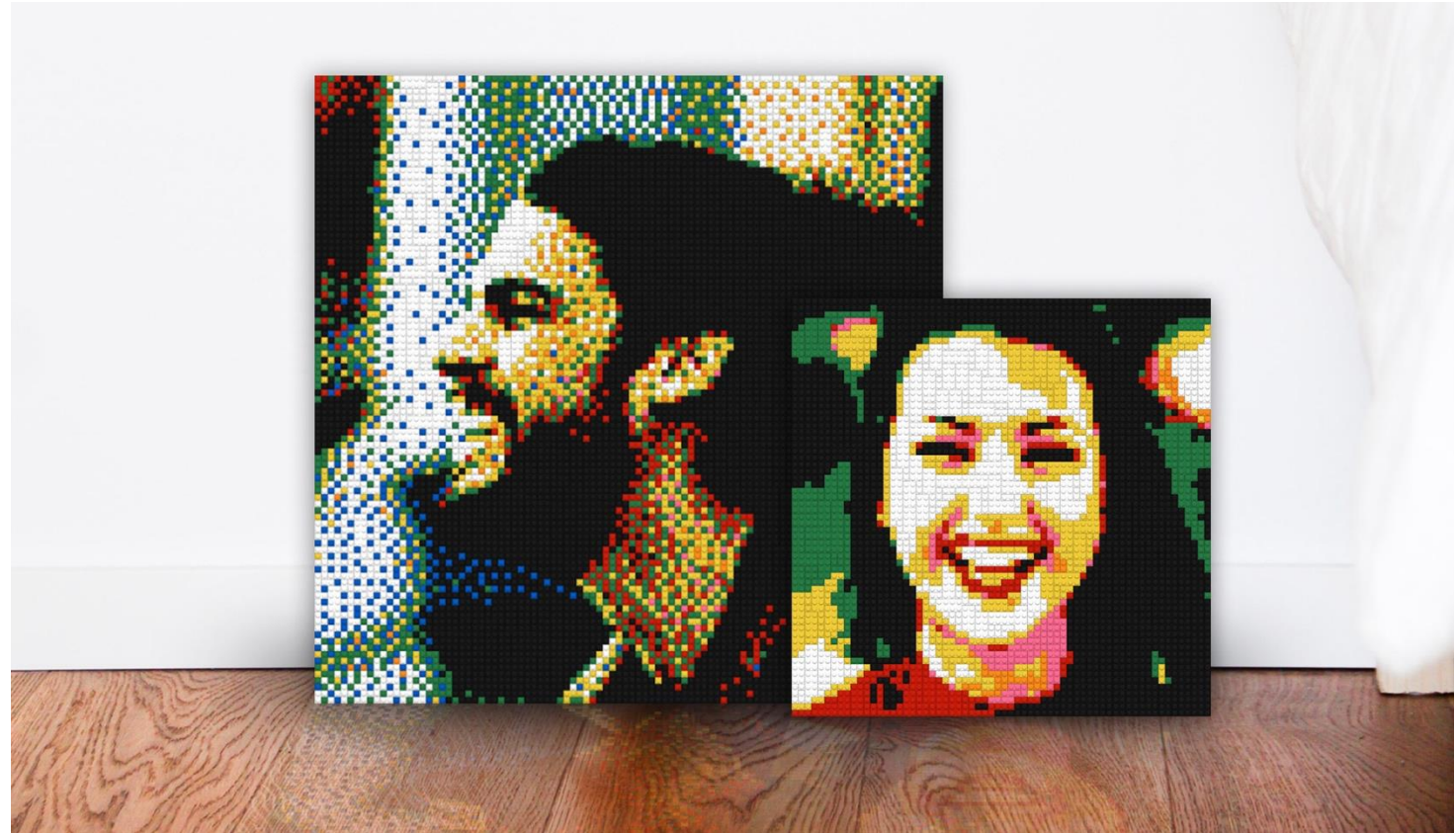
Can we do this using
Morphological
Operations?



<https://brickmypicture.com/>



Can we do this using
Morphological
Operations?



<https://brickmypicture.com/>



After-class Reflection and Beyond

- 1. Why is edge detection useful for understanding the content of an image?**
Give one example where detecting object boundaries could help solve a computer vision problem.
- 2. An edge in an image may be caused by changes in depth, surface orientation, material, or illumination.**
Can you think of an example where an edge does **not** correspond to a real object boundary?
- 3. Why does the output image become smaller when we apply a 3×3 filter to an image without padding?**
How does padding help preserve the original image size?
- 4. Why do we often need to normalize filter outputs before displaying them as images?**
What might happen if we directly display an edge-filtered image without normalization?
- 5. The Laplacian filter is sensitive to noise.**
Why might applying Gaussian smoothing before Laplacian edge detection improve the result?
- 6. Morphological operations use a structuring element instead of a convolution kernel.**
What is the main difference between convolution-based filtering and morphology-based processing?
- 7. Erosion tends to shrink foreground objects, while dilation tends to expand them.**
Explain this behavior using the concepts of “fit” and “hit.”
- 8. Opening is erosion followed by dilation, while closing is dilation followed by erosion.**
Give one practical situation where opening would be useful, and one practical situation where closing would be useful.



Thank You!